

EVALUATING THE IMPACT OF CONTAINERIZATION ON REAL-TIME TASK PREDICTABILITY IN LINUX SYSTEMS

Isaac Stephens

Junior, Computer Science, CS Department, Missouri S&T [issq3r@mst.edu]

Abstract – This project evaluates the effect of containerization on soft real-time task predictability in Linux systems. The study compares periodic CPU-bound workloads executed directly on the host system and inside Docker containers under both a standard mainline Linux kernel and a PREEMPT_RT kernel. The workloads were generated using rt-app and evaluated using metrics such as cyclicttest maximum latency, rt-app deadline miss rate, rt-app jitter, and rt-app average wake-up latency. A bare-metal mainline CFS configuration was also included as a baseline comparison. Across the collected results, the PREEMPT_RT kernel generally produced much lower cyclicttest latency than the standard mainline kernel, suggesting improved scheduling responsiveness. Docker did not always severely degrade rt-app behavior, but containerized runs still showed evidence of added timing variability, especially under the mainline kernel. The results also showed that deadline miss behavior was strongly affected by workload utilization and task structure, not only by the execution environment. Because the experiment was performed on a non-isolated laptop rather than a dedicated test machine, the results should be interpreted as exploratory evaluation rather than a complete characterization of Linux real-time performance. Overall, the study suggests that PREEMPT_RT can improve latency predictability, while containerization may be practical for soft real-time workloads when system configuration and workload limits are carefully controlled.

1 INTRODUCTION

Modern distributed systems increasingly deploy workloads using container-based virtualization. Containers are widely used in edge computing, industrial control gateways, robotics, and cloud-based control pipelines. Many of these systems operate under soft real-time constraints, where bounded latency and low jitter are required, but occasional deadline misses may be tolerated.

For such applications Linux-based operating systems are commonly chosen due to their flexibility, ecosystem maturity, and maintainability. However, the Linux kernel does not provide hard real-time guarantees by default. Although it supports real-time scheduling classes such as SCHED_FIFO and SCHED_DEADLINE, worst-case latency is not strictly bounded under the default preemption model [1]. Furthermore, container platforms such as Docker, LXC, and Kubernetes share the host kernel and introduce additional scheduling abstractions through namespaces and cgroups. These abstractions may affect scheduling behavior and timing predictability.

This work will evaluate soft real-time predictability through the implementation of a periodic real-time workload generator on Linux to measure task latency, jitter, and deadline miss behavior using high-resolution timers. The study compares performance of the SCHED_DEADLINE scheduling policy under two kernel configurations: (1) the standard mainline kernel operating under its default preemption model, and (2) a kernel configured with the PREEMPT_RT patch to enable fully preemptible kernel behavior. For each kernel configuration, experiments are conducted in both bare-metal and containerized (Docker) environments. This comparison isolates the impact of kernel-level preemption behavior and container-level resource isolation on soft real-time predictability.

The central research question of this work is: To what extent does containerization degrade latency, jitter, and deadline adherence compared to bare-metal execution, and can these effects be mitigated through system-level tuning?

2 BACKGROUND

2.1 REAL-TIME SCHEDULING IN LINUX

The Linux kernel provides multiple scheduling classes to support general-purpose and real-time workloads. The default scheduling class, `SCHED_NORMAL`, is implemented by the Completely Fair Scheduler (CFS), which aims to distribute CPU time proportionally among runnable tasks. While CFS is designed for fairness and throughput, it does not provide deterministic timing guarantees and is therefore unsuitable for strict real-time constraints.

To support time-sensitive workloads, Linux provides fixed-priority real-time scheduling classes, including `SCHED_FIFO` and `SCHED_RR`. Under these policies, tasks with higher static priority preempt lower-priority tasks, enabling bounded response times under controlled conditions. However, even when real-time policies are used, the standard mainline kernel contains non-preemptible sections, interrupt handling latencies, and kernel locking mechanisms that can introduce unbounded worst-case delays [1].

More recently, Linux introduced the `SCHED_DEADLINE` scheduling class, which implements an Earliest Deadline First (EDF) scheduler combined with a constant Bandwidth Server (CBS) mechanism, enabling temporal isolation among competing real-time tasks. While EDF and CBS provide strong theoretical guarantees on schedulability when total utilization is bounded, practical behavior in the Linux kernel implementation can deviate from theory under certain conditions such as task affinities and kernel interactions. Prior work has identified implementation limitations in the Linux `SCHED_DEADLINE` scheduler with respect to tardiness bounds and affinity behavior, highlighting that theoretical EDF guarantees do not always hold in practice on mainstream kernels [2].

Despite these theoretical properties, practical timing behavior in Linux heavily on kernel preemption behavior and interrupt handling. The `PREEMPT_RT` patch modifies the Linux kernel to become fully preemptible. It reduces non-preemptible sections, converts most interrupt handlers into preemptible kernel threads, and introduces priority inheritance mechanisms to mitigate priority inversion. Prior evaluation have shown that `PREEMPT_RT` significantly reduces worst-case scheduling latency compared to the default kernel configuration [1]. However, even under `PREEMPT_RT`, strict hard real-time guarantees require careful system configuration and workload isolation.

2.2 CONTAINERIZATION AND RESOURCE ISOLATION

In Linux container environments, isolation between applications is primarily implemented using namespaces and control groups (`cgroups`). Container virtualization evolved from earlier process isolation mechanisms such as the `chroot` system call and later systems like FreeBSD Jails, which allowed multiple isolated environments to run on a single operating system instance [4]. Modern Linux container implementations build upon these ideas by providing kernel-level primitives that enable processes to operate in isolated execution contexts while still sharing the same host kernel. Namespaces provide logical separation of kernel resources so that processes inside a container perceive a private instance of system components such as process identifiers, network interfaces, mount points, and inter-process communication mechanisms [3].

While namespaces isolate the system view presented to processes, `cgroups` provide mechanisms for resource accounting and enforcement. Control groups allow the Linux kernel to monitor and limit the usage of system resources such as CPU time, memory, and I/O bandwidth for groups of processes. Container runtimes such as Docker and LXC configure namespaces and `cgroups` together to create isolated

execution environments for applications running on a shared kernel. Because containers rely on the host kernel scheduler for CPU allocation, the interaction between `cgroup` resource controls and Linux scheduling policies can influence the timing behavior of real-time workloads [3].

Resource isolation in containerized systems is often implemented using `cgroup`-based CPU bandwidth control mechanisms that limit the amount of processor time available to groups of tasks. While this approach allows the operating system to enforce resource quotas across containers, prior work has shown that `cgroup`-based CPU control may not always provide strong temporal isolation under high system load. In particular, experiments comparing `cgroups` with the `SCHED_DEADLINE` scheduling policy demonstrate that tasks relying solely on `cgroup` bandwidth control can experience significant interference from co-located workloads, leading to delayed execution and reduced CPU bandwidth availability. In contrast, tasks scheduled using `SCHED_DEADLINE` are able to maintain CPU bandwidth closer to their requested allocation, resulting in substantially smaller execution delays and more predictable response times. These findings suggest that real-time scheduling policies may provide stronger temporal isolation for critical workloads than `cgroup`-based mechanisms alone, particularly under high CPU utilization conditions [5].

2.3 REAL-TIME PERFORMANCE OF CONTAINERS

Recent research has investigated whether container-based virtualization can support workloads with real-time constraints. Several studies have evaluated container performance using metrics such as latency, jitter, and deadline adherence under controlled workloads. These studies generally find that containers introduce relatively small overhead compared to native execution because they share the host operating system kernel and avoid the full hardware abstraction layer used in traditional virtual machines. As a result, containerized environments can often achieve comparable average performance while enabling flexible deployment and modular system architectures [6].

However, despite the low overhead of containerization, maintaining predictable real-time behavior remains challenging. Prior work has shown that containerized workload can experience increased timing variability when multiple containers compete for CPU resources or when resource management mechanisms such as `cgroups` introduce scheduling interference. In such cases, the timing behavior of real-time tasks may degrade due to contention for shared kernel resources and delays introduced by background workloads [7].

Empirical evaluations of containerized real-time systems have also examined their suitability for industrial automation and cyber-physical applications. Some studies demonstrate that containerized deployments can meet soft real-time deadlines in distributed control systems and microservice-based architectures when appropriate scheduling policies and system configurations are used [8]. Other work evaluating virtualization technologies for real-time workloads emphasizes that worst case-latency and temporal isolation remain key challenges when deploying time-sensitive applications in shared computing environments [9].

More recent experimental work has focused on quantifying latency, jitter, and resource consumption across different virtualization platforms and deployment configurations. These studies highlight the importance of kernel configuration, scheduling policies, and resource allocation mechanisms in determining whether containerized systems can achieve predictable real-time behavior [10]. These findings motivate the experimental evaluation conducted in this work, which compares the behavior of real-time tasks across bare-metal and containerized environments under different Linux kernel configurations.

3 METHODOLOGY / METRICS

3.1 EXPERIMENTAL APPROACH AND COMPARISON METHOD

This work evaluates how containerization affects soft real-time predictability in Linux by executing the same periodic real-time workload under multiple system configurations and comparing timing outcomes. The workload is scheduled using the Linux `SCHED_DEADLINE` scheduling class and is executed in two execution environments: (1) directly scheduled by the kernel or “bare-metal” and (2) Docker containers. Each environment is tested under two kernel configurations: (a) a standard mainline Linux kernel, Linux 6.12.57+deb13-amd64, using the default preemption model, and (b) a kernel re-configured with the `PREEMPT_RT` patch. This design isolates the impact of container-level isolation mechanism and kernel-level preemption behavior on timing predictability. All other factors are controlled as much as possible across configurations, including CPU affinity, workload parameters, experiment duration, and background services.

3.2 WORKLOAD GENERATION

The periodic workload used in this study is generated using `rt-app`, a workload generation tool designed for evaluating real-time scheduling behavior in Linux systems [11]. It allows the creation of periodic tasks with explicitly defined real-time parameters, including execution runtime C , activation period T , and relative deadline D . These parameters correspond directly to the classical periodic real-time task model used in scheduling theory, where task utilization is defined as $U = C/T$.

In this study, `rt-app` is used to generate periodic CPU-bound workloads representative of soft real-time applications such as edge computing services, robotics middleware components, and industrial control tasks. The workloads are executed under identical parameters across all experimental configurations to isolate the effects of containerization and kernel preemption behavior on real-time performance.

3.3 PERFORMANCE MEASUREMENT AND METRICS

To evaluate real-time predictability under each experimental configuration, timing measurements are collected using both the `rt-app` workload generator and the Linux real-time latency measurement tool `cyclicttest`. `cyclicttest` is widely used in real-time Linux evaluations to measure scheduling latency by repeatedly waking a periodic task and recording the delay between the expected activation time and the actual wake-up time [12]. In this study, `cyclicttest` is used to establish a baseline measurement of system scheduling latency under each kernel configuration and execution environment. These measurements provide an independent reference for understanding the underlying scheduling behavior of the system before running the full workload experiments.

In addition to `cyclicttest` measurements, the periodic tasks generated by `rt-app` record timing information using high-resolution monotonic clocks available in the Linux kernel. For each job release, the workload records three timestamps: the expected release time, the actual start time when the task begins executing, and the completion time when the job finishes execution. These timestamps are logged for offline analysis and are used to compute latency, jitter, and deadline miss rate.

These three metrics are used to evaluate scheduling predictability. Latency L_k is defined as the difference between the expected release time r_k and the observed start time s_k of the task k such that $L_k = s_k - r_k$, representing the delay introduced by the operating system scheduler on competing workloads. Jitter J refers to the variability in latency across successive job executions and is calculated as the standard deviation of the observed latency values. High jitter indicates unstable scheduling behavior and reduced timing predictability. Deadline misses occur when the completion time of a job exceeds its relative deadline D , meaning the task fails to complete within its required execution window. By analyzing these metrics across

all experimental configurations, the study quantifies how containerization and kernel preemption models influence the predictability of periodic real-time workloads.

4 RESULTS AND DISCUSSION

4.1 PLOTS & FIGURES

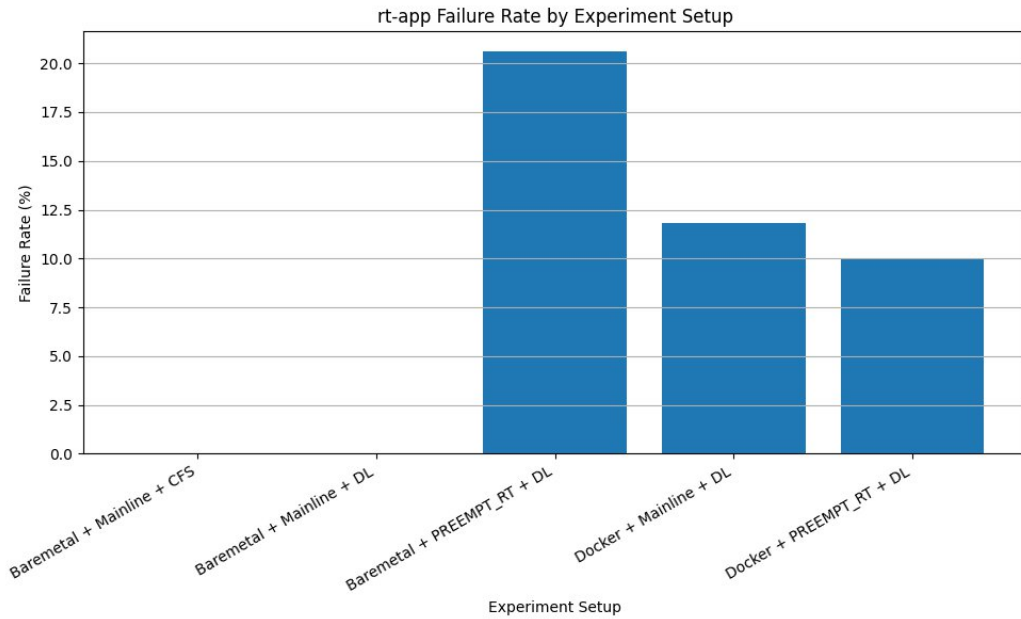


Figure. 1: Failure Rate of `rt-app` by Experiment Setup

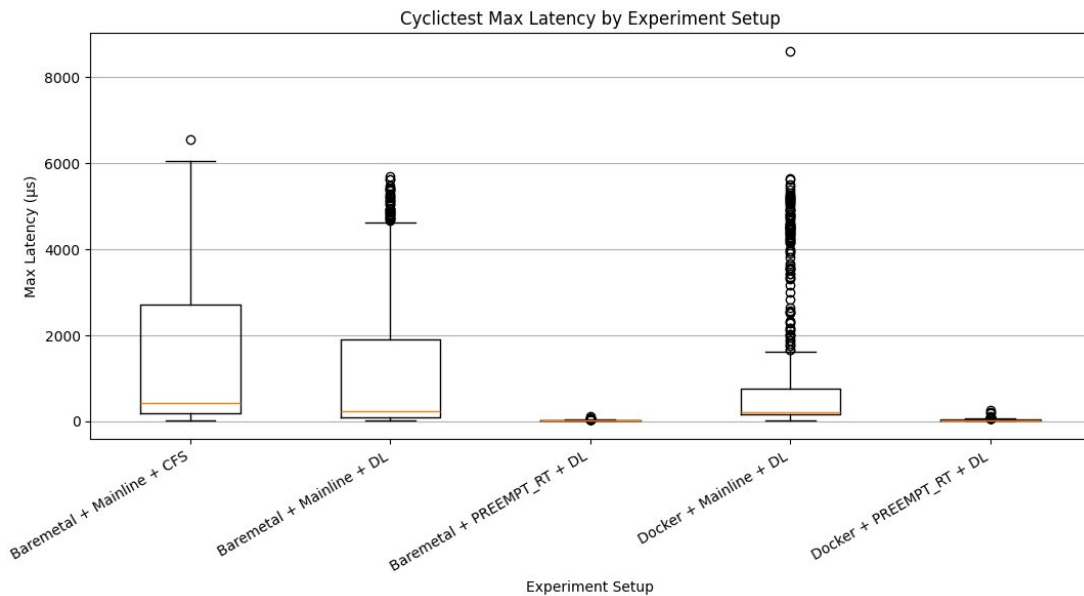


Figure. 2: `cyclictest` Max Latency Box-plot by Setup

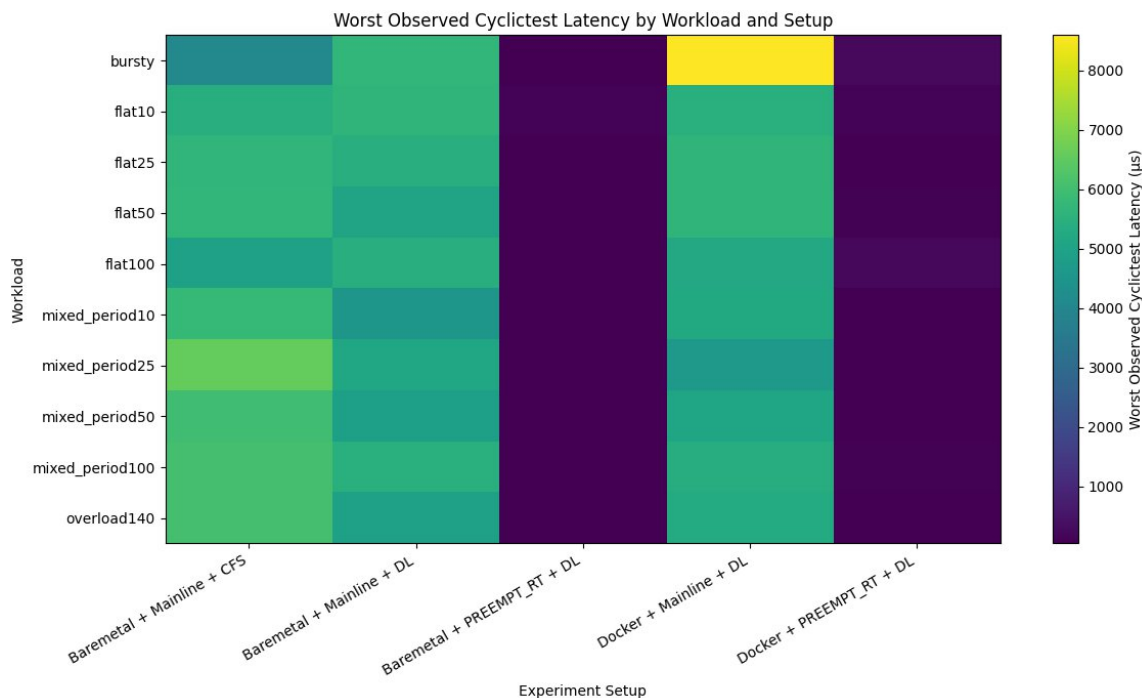


Figure. 3: Worst Observed cyclicttest Latency Heat-map by Workload & Setup

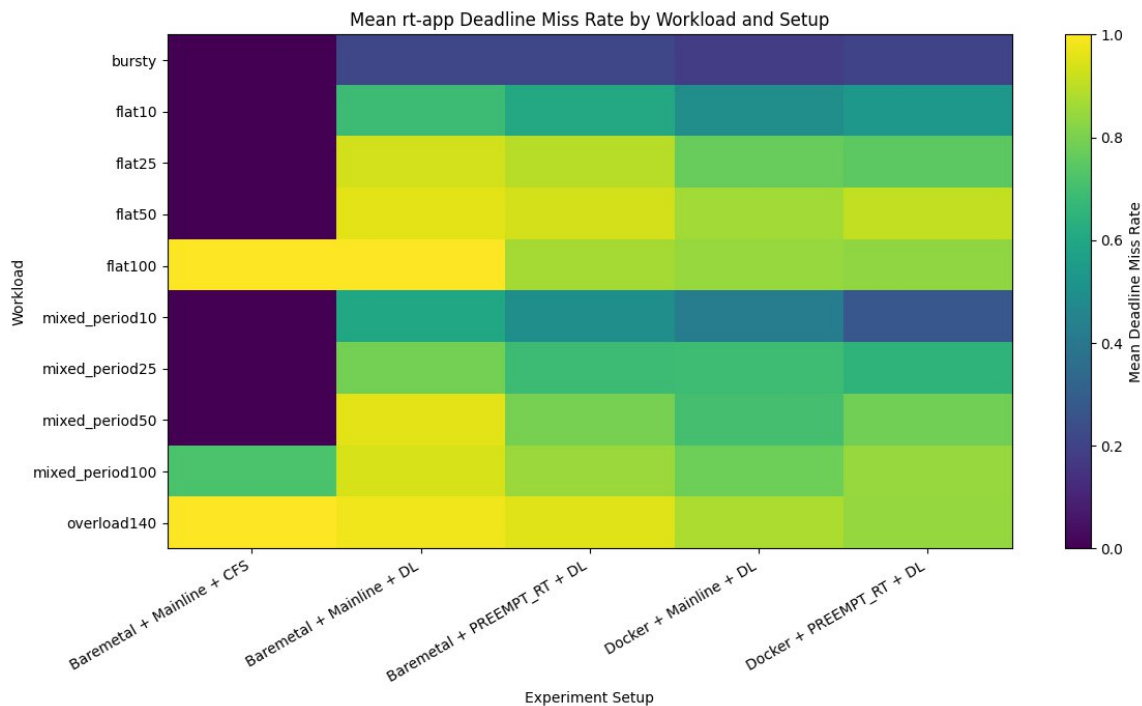
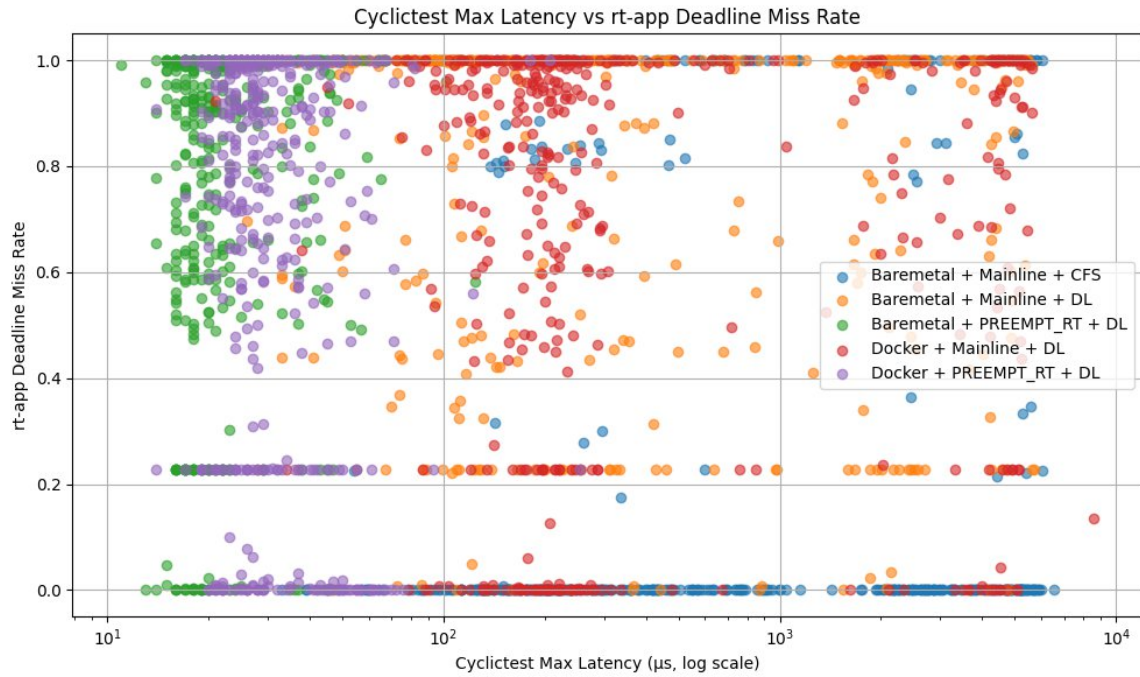


Figure. 4: Mean rt-app Deadline Miss Rate Heat-map by Workload & Setup

Table 1: Median $rt\text{-app}$ Jitter (μs) by Workload & Setup

Setup	Baremetal + Mainline + CFS	Baremetal + Mainline + DL	Baremetal + PREEMPT_RT + DL	Docker + Mainline + DL	Docker + PREEMPT_RT + DL
bursty	8653030	4.83064	44.3423	47.5944	38.26225
flat10	8645800	2.095875	28.3151	35.62355	29.698
flat25	8645810	0	16.3309	14.8659	16.7092
flat50	8645820	0	3.5901	3.23112	3.3141
flat100	8661825	0	0	0	0
mixed_period10	8645810	33636.5	26859.9	23692.3	49.6407
mixed_period25	8645810	7784.495	16043.05	15216.3	17180.9
mixed_period50	8645790	0	4751.24	4623.02	5539.545
mixed_period100	8658280	0	0.310685	0	0
overload140	8661000	0	0	0	0

Figure. 5: cyclictest Max Latency vs $rt\text{-app}$ Miss Rate

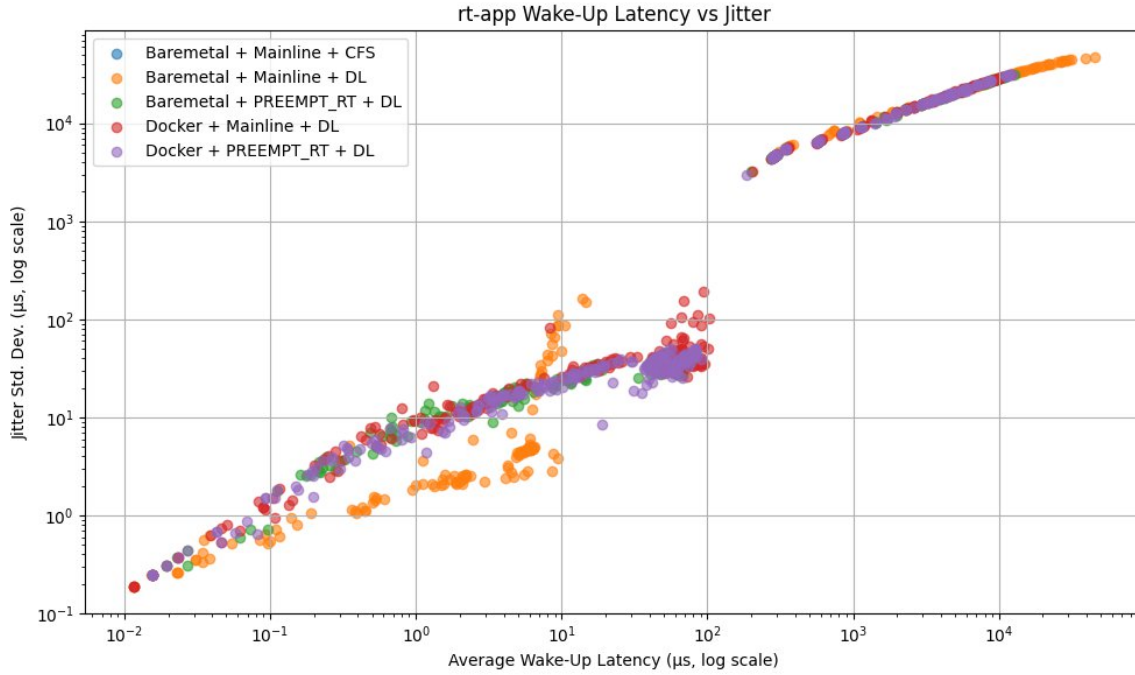


Figure 6: Scatter Plot of `rt-app` Wake-up Latency vs Jitter

4.2 DISCUSSION

The results show that kernel configuration had a noticeable effect on scheduling latency. Figures 2 and 3 show that the `PREEMPT_RT` configurations generally produced much smaller `cyclicttest` maximum latency values than the mainline kernel configurations. This trend appears in both the bare-metal and Docker environments. In the appendix, the worst observed `cyclicttest` latencies for the mainline kernel configurations are generally in the thousands of microseconds, while the `PREEMPT_RT` configurations are usually much lower. For example, the bare-metal `PREEMPT_RT + SCHED_DEADLINE` setup reports worst observed `cyclicttest` latencies such as $47\ \mu\text{s}$ for `flat25`, and $39\ \mu\text{s}$ for `mixed_period100`. In contrast, the bare-metal mainline configurations often report worst case observed `cyclicttest` latencies above $5000\ \mu\text{s}$ for the same general workload group. This suggests that `PREEMPT_RT` improved the responsiveness of the system by reducing scheduling delay, which is consistent with the expected purpose of the real-time kernel patch.

The Docker results were more mixed. Docker + Mainline + `SCHED_DEADLINE` often showed `cyclicttest` latency values similar to the bare-metal mainline configurations, and in some cases the worst observed latency was higher. For example, the bursty workload reached a worst observed `cyclicttest` latency of $8594\ \mu\text{s}$ in the Docker + Mainline + `SCHED_DEADLINE` setup. This suggests that container execution under the mainline kernel may introduce additional timing variability, or at least does not remove the underlying latency behavior of the host kernel. However, Docker + `PREEMPT_RT` + `SCHED_DEADLINE` showed much lower `cyclicttest` latency for the bursty workload than Docker on the mainline kernel at $254\ \mu\text{s}$. This indicates that the host kernel configuration still plays a major role in containerized real-time behavior because Docker containers share the host kernel rather than running an independent kernel like a dedicated virtual machine would [6].

Figure 1 and Table 3 show that `rt-app` failures were not distributed evenly across the experiment setups. The baseline Bare-metal + Mainline + CFS and Bare-metal + Mainline + `SCHED_DEADLINE` had no

recorded `rt-app` failures. However, Bare-metal + `PREEMPT_RT` + `SCHED_DEADLINE` had a 20.6% failure rate, Docker + Mainline + `SCHED_DEADLINE` had an 11.8% failure rate, and Docker + `PREEMPT_RT` + `SCHED_DEADLINE` had a 10.0% failure rate. These failures were caused by a divide-by-zero exception inside `rt-app`. Because the failure occurred inside the workload tool rather than in the data analysis script, failed runs were logged and excluded from the final metric calculations. This limits the strength of the conclusions because the successful trials may not fully represent every attempted run. However, excluding failed trials was necessary to avoid mixing invalid or incomplete timing logs into the final statistics.

The deadline miss results in Figure 4 show that deadline adherence was affected by more than just the kernel or container environment. Workload utilization and workload structure were also important. The higher-utilization workloads, especially `flat100` and `overload140`, produced high miss rates across multiple configurations. This is expected because these workloads place the system near or above full utilization. In the `overload140` case, total CPU utilization is 140%, which is intentionally above the feasible utilization bound for a single processor when all tasks require their full execution time. Therefore, high miss rates in this workload should not be interpreted as only a failure of Docker or the kernel. Instead, they show how the system behaves under overload conditions.

Table 1 shows that `rt-app` jitter also varied strongly by workload type. The mixed-period workloads often produced larger jitter values than the flat-period workloads. This is reasonable because mixed-period task sets create more varied release patterns and more complex scheduling interactions than task sets where every task has the same period. For example, `mixed_period10` and `mixed_period25` show relatively large jitter values in several `SCHED_DEADLINE` configurations, while `flat50`, `flat100`, and `overload140` often show median jitter values near zero in some configurations. This does not mean those workloads performed better overall, because some of them still had high deadline miss rates, approaching 100%. Instead, it shows that jitter and deadline miss rate measure different aspects of predictability. A workload can have low measured jitter while still missing deadlines if it is overloaded or if the task timing pattern consistently fails to complete before the deadline.

The CFS baseline is useful as a general comparison point, but it should be interpreted carefully. The baseline used `SCHED_OTHER/CFS` rather than `SCHED_DEADLINE`, so it was not expected to provide the same deadline-aware behavior as the main experiment configurations. The CFS baseline also did not include the `rt-app` average wake-up latency metric due to time constraints, which is why it does not appear in Figure 6. The `rt-app` jitter metric was still collected for the baseline, but the baseline values are much larger than the `SCHED_DEADLINE` values. This suggests that CFS is not well suited for predictable periodic real-time execution. However, because the baseline was not configured with the same scheduling policy as the other setups, it should mainly be treated as a reference for normal Linux scheduling behavior rather than a direct competitor to the `SCHED_DEADLINE` configurations.

Figure 5 compares `cyclicttest` maximum latency with `rt-app` deadline miss rate. The relationship between these two metrics does not appear to be perfectly linear. Some configurations may have relatively low `cyclicttest` latency but still show high miss rates for specific workloads. This is because `cyclicttest` measures wake-up latency using a controlled periodic test thread, while `rt-app` measures behavior under the actual workload configuration. Deadline misses depend on task execution time, period, deadline, utilization, scheduler behavior, and interference from other tasks. Therefore, `cyclicttest` is useful for understanding baseline scheduling latency, but it does not fully predict whether a specific workload will meet every deadline.

Figure 6 compares `rt-app` average wake-up latency with `rt-app` jitter for the `SCHED_DEADLINE` configurations. The CFS baseline does not appear in this plot because average wake-up latency was not collected for that setup. Because the figure is plotted on a log scale, the relationship is easier to see across workloads with very different timing magnitudes. The plotted data suggests a positive curved relationship

between wake-up latency and jitter, approximately following a $y = \sqrt{x}$ -shaped trend for much of the data. In general, lower average wake-up latency coincides with lower jitter, while higher wake-up latency corresponds to increased jitter.

One notable feature is that around an average wake-up latency of approximately $10^2 \mu s$, the jitter values appear to jump from roughly $10^2 \mu s$ to approximately $10^3 \mu s$ across the tested environments. After this jump the trend continues in a more expected curved pattern. This may suggest a threshold region where increased wake-up delay begins to have a stronger effect on timing variability.

There were several important limitations in this experiment. First, the tests were performed on a laptop that was still being used as a normal operating system environment during the testing phase. Background processes, desktop environment activity, interrupts, power management behavior, and other system services may have affected timing measurements. A dedicated clean test machine with minimized background activity would likely produce cleaner and more repeatable results. Second, the number of experiment configurations was limited by time. The project compared mainline and `PREEMPT_RT` kernels, bare-metal and Docker execution, and several workload utilization levels, but it did not deeply explore every possible tuning option. Third, the Docker configuration required practical adjustments. Docker CPU-set restrictions caused `SCHED_DEADLINE` to fail with an “Operation not permitted” error when using Docker’s `--cpuset-cpus` option. Removing that option allowed `rt-app` to apply the deadline scheduler successfully, suggesting that CPU affinity for the Docker runs should be controlled inside the `rt-app` workload configuration rather than through Docker’s `cpuset` cgroup.

Overall, the results suggest that `PREEMPT_RT` improved scheduling latency, especially as measured by `cyclicttest`. Containerization did not eliminate the benefits of `PREEMPT_RT`, since Docker + `PREEMPT_RT` still showed much lower `cyclicttest` latency than Docker + Mainline. However, the `rt-app` results also show that real-time predictability depends heavily on workload utilization, task period structure, and system stability. For soft real-time systems, the results suggest that containers can be usable, but only when the host kernel, scheduling policy, workload utilization, and CPU affinity are configured carefully.

5 CONCLUSIONS

This project showed that kernel configuration has a major effect on soft real-time predictability in Linux. The `PREEMPT_RT` kernel generally produced much lower scheduling latency than the standard mainline kernel, especially in the `cyclicttest` results. This suggests that improving kernel preemption behavior is one of the most useful steps for reducing latency in Linux-based real-time workloads.

The results also showed that containerization does not automatically prevent soft real-time execution. Docker workloads were still able to run under `SCHED_DEADLINE`, and Docker with `PREEMPT_RT` showed much better latency behavior than Docker on the mainline kernel. However, the containerized results also suggest that Docker adds configuration complexity and may contribute to timing variability if the system is not carefully controlled.

A major lesson from the experiment is that real-time predictability depends on more than one metric. Wake-up latency, jitter, `cyclicttest` latency, and deadline miss rate each describe different parts of the system’s behavior. Workload structure and utilization also mattered, especially for the mixed-period and overloaded workloads.

Overall, the study suggests that `PREEMPT_RT` can improve soft real-time predictability and that containerized real-time workloads are possible, but only with careful scheduling, workload design, and system configuration. Future work should repeat the experiment on a dedicated test machine with better isolation to reduce interference from background operating system activity.

6 FUTURE WORK

Future work should repeat the experiment on a dedicated test machine with minimal background activity to reduce interference from the operating system and improve repeatability. Additional experiments could also evaluate different CPU-affinity and `cgroup` configurations to determine how resource isolation affects real-time predictability in Docker. Finally, testing a larger range of workloads, utilization levels, and container configurations would provide a more complete understanding of the conditions which containerized soft real-time workloads can maintain predictable timing.

7 REFERENCES

- [1] F. Reghenzani, G. Massari, and W. Fornaciari, “The Real-time Linux Kernel,” *ACM Computing Surveys*, vol. 52, no. 1, pp. 1-36, Feb. 2019, doi: <https://doi.org/10.1145/3297714>.
- [2] S. Tang, J. H. Anderson, and Luca Abeni, “On the Defectiveness of SCHED_DEADLINE w.r.t. Tardiness and Affinities, and a Partial Fix,” pp. 46–56, Apr. 2021, doi: <https://doi.org/10.1145/3453417.3453440>.
- [3] L. Abeni, A. Balsini, and T. Cucinotta, “Container-based real-time scheduling in the Linux kernel,” *ACM SIGBED Review*, vol. 16, no. 3, pp. 33–38, Nov. 2019, doi: <https://doi.org/10.1145/3373400.3373405>.
- [4] R. Queiroz, T. Cruz, J. Mendes, P. Sousa, and Paulo Simões, “Container-based Virtualization for Real-Time Industrial Systems – A Systematic Review,” *ACM Computing Surveys*, Aug. 2023, doi: <https://doi.org/10.1145/3617591>.
- [5] M. Thiyyakat, S. Kalambur, and D. Sitaram, “Improving Resource Isolation of Critical Tasks in a Workload.” Accessed: Mar. 04, 2026. [Online]. Available: https://jsspp.org/papers20/Improving-Resource-Isolation_JSSPP2020.pdf?utm_source=google.com
- [6] Václav Struhár, M. Behnam, M. Ashjaei, and Alessandro Vittorio Papadopoulos, “Real-time containers : A survey,” vol. 80, p. 9, Jan. 2020, doi: <https://doi.org/10.4230/oasics.fog-iot.2020.7>.
- [7] Václav Struhár, S. S. Craciunas, M. Ashjaei, M. Behnam, and Alessandro Vittorio Papadopoulos, “REACT: Enabling Real-Time Container Orchestration,” Zenodo (CERN European Organization for Nuclear Research), Sep. 2021, doi: <https://doi.org/10.1109/etfa45728.2021.9613685>.
- [8] D. Martínez, S. L. Castaño, J. C. Perafan, and O. H. Mondragón, “Microservices-Based Architecture to Support Automation Applications in Smart Industries with End-to-End Real-Time Requirements,” *Tsinghua Science and Technology*, vol. 31, no. 2, pp. 708–725, Apr. 2026, doi: <https://doi.org/10.26599/tst.2024.9010235>.
- [9] Luca Abeni, “Virtualized real-time workloads in containers and virtual machines,” *Journal of Systems Architecture*, vol. 154, pp. 103238–103238, Jul. 2024, doi: <https://doi.org/10.1016/j.sysarc.2024.103238>.
- [10] N. Ben Kebaier, B. Geib, E. Lyczkowski, R. Venanzi, and M. Barth, “Real-Time Performance Evaluation of Containerized Virtual PLCs: A Comparative Study of Docker and Podman for Industrial Automation,” 2025 IEEE 30th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD), pp. 1–6, Oct. 2025, doi: <https://doi.org/10.1109/camad67323.2025.11229890>.
- [11] Shah, R. G. Shenoy, V. Srinivasan, P. Bose, and A. Buyuktosunoglu, “TokenSmart: Distributed, Scalable Power Management in the Many-core Era,” *ACM Transactions on Architecture and Code Optimization*, vol. 20, no. 1, pp. 1–26, Nov. 2022, doi: <https://doi.org/10.1145/3559762>.
- [12] D. Bristot De Oliveira, D. Casini, R. Silva De Oliveira, and T. Cucinotta, “Demystifying the Real-Time Linux Scheduling Latency.” Accessed: Mar. 05, 2026. [Online]. Available: https://retis.santannapisa.it/~tommaso/publications/ECRTS-2020.pdf?utm_source=google.com.

CpE 5170 REAL TIME SYSTEMS

Final Report

page 13

8 APPENDIX

8.1 ADDITIONAL TABLES

Table 2: Summary of All Measured Results

Setup	Workload	# of Trials	Mean Jitter (µs)	Median Jitter (µs)	Jitter Stddev (µs)	Min Jitter (µs)	Max Jitter (µs)	Mean Miss Rate	Mean Cyclictest Latency	Max Cyclictest Latency
Baremetal + Mainline + CFS	bursty	50	8.65E+06	8.65E+06	5.095016	8.65E+06	8.65E+06	0	925.08	4057
	flat10	50	8.65E+06	8.65E+06	618.151458	8.64E+06	8.65E+06	0	1901.78	5426
	flat100	50	8.66E+06	8.66E+06	11959.07555	8.64E+06	8.68E+06	1	1522.58	4920
	flat25	50	8.65E+06	8.65E+06	346.922858	8.64E+06	8.65E+06	0	1424.92	5662
	flat50	50	8.65E+06	8.65E+06	114.611856	8.65E+06	8.65E+06	0	1723.02	5720
	mixed period10	50	8.65E+06	8.65E+06	9.64894	8.65E+06	8.65E+06	0	1330.66	5759
	mixed period100	50	8.66E+06	8.66E+06	11653.12973	8.64E+06	8.68E+06	0.715754	1511.26	6063
	mixed period25	50	8.65E+06	8.65E+06	4083.740602	8.65E+06	8.67E+06	0	2126.92	6557
	mixed period50	50	8.65E+06	8.65E+06	4105.7593	8.64E+06	8.67E+06	0	1599.48	5971
	overload140	50	8.66E+06	8.66E+06	15737.91074	8.63E+06	8.70E+06	1	1801.64	6056
Baremetal + Mainline + DL	bursty	50	2.18E+01	4.83E+00	37.392563	3.04E+00	1.61E+02	0.213698	1186.86	5702
	flat10	50	3.53E+02	2.10E+00	1394.681443	0.00E+00	6.01E+03	0.687146	1184.34	5633
	flat100	50	0.00E+00	0.00E+00	0	0.00E+00	0.00E+00	1	1223.22	5414
	flat25	50	1.53E+00	0.00E+00	7.893573	0.00E+00	5.60E+01	0.93073	1398.84	5422
	flat50	50	1.32E+01	0.00E+00	0.662513	0.00E+00	3.82E+00	0.960468	1280.32	5047
	mixed period10	50	2.74E+04	3.36E+04	15586.35845	0.00E+00	4.71E+04	0.594957	829.08	4570
	mixed period100	50	5.24E+01	0.00E+00	3.014968	0.00E+00	2.11E+01	0.940468	1233.22	5476
	mixed period25	50	1.08E+04	7.78E+03	10576.20073	0.00E+00	2.66E+04	0.790097	1656.02	5124
	mixed period50	50	3.59E+02	0.00E+00	1489.382953	0.00E+00	8.32E+03	0.957661	704.58	4872
	overload140	50	4.77E+02	0.00E+00	0.337038	0.00E+00	2.38E+00	0.98	874.88	4943
Baremetal + PREEMPT_RT + DL	bursty	40	4.28E+01	4.43E+01	4.382036	2.90E+01	4.77E+01	0.211325	21.6	47
	flat10	45	2.68E+02	2.83E+01	1131.913789	8.83E+00	5.50E+03	0.604357	23.266667	124
	flat100	39	5.10E+00	0.00E+00	12.598049	0.00E+00	3.96E+01	0.865298	24.307692	64
	flat25	35	1.61E+01	1.63E+01	3.163612	1.02E+01	2.19E+01	0.893925	22.942857	46
	flat50	37	5.62E+00	3.59E+00	7.14566	0.00E+00	3.18E+01	0.934059	21.324324	46
	mixed period10	41	2.37E+04	2.69E+04	9223.770976	2.71E+01	3.17E+04	0.496316	25	57
	mixed period100	41	8.49E+01	3.11E+01	500.756029	0.00E+00	3.21E+03	0.849169	20.317073	39
	mixed period25	36	1.51E+04	1.60E+04	5973.032017	2.73E+01	2.49E+04	0.685275	25.166667	65
	mixed period50	41	5.20E+03	4.75E+03	3556.669495	1.03E+01	1.19E+04	0.795307	23.414634	48
	overload140	42	1.74E+00	0.00E+00	7.880367	0.00E+00	3.88E+01	0.952381	31.428571	67
Docker + Mainline + DL	bursty	41	5.19E+01	4.76E+01	20.48093	2.95E+01	1.55E+02	0.181327	1499.219512	8594
	flat10	46	3.96E+02	3.56E+01	1379.031753	2.60E+01	5.67E+03	0.492457	1167.130435	5453
	flat100	45	5.78E+00	0.00E+00	13.55903	0.00E+00	3.92E+01	0.844364	882.488889	5207
	flat25	45	1.94E+01	1.49E+01	13.512393	6.83E+00	8.12E+01	0.769209	1013.111111	5632
	flat50	47	8.24E+00	3.23E+00	12.115897	0.00E+00	4.20E+01	0.863195	1084.553191	5641
	mixed period10	41	1.93E+04	2.37E+04	10770.65267	2.86E+01	3.11E+04	0.424257	1119.04878	5232
	mixed period100	46	8.13E+01	0.00E+00	480.415725	0.00E+00	3.27E+03	0.781119	973.934783	5384
	mixed period25	46	1.41E+04	1.52E+04	6101.840284	2.92E+01	2.24E+04	0.687623	1048.26087	4674
	mixed period50	43	4.32E+03	4.62E+03	3465.93525	9.45E+00	9.45E+03	0.704004	950.534884	5087
	overload140	41	8.56E+00	0.00E+00	31.496545	0.00E+00	1.90E+02	0.878049	1319.878049	5319
Docker + PREEMPT_RT + DL	bursty	50	3.88E+01	3.83E+01	5.178294	2.26E+01	5.07E+01	0.202355	37.36	254
	flat10	45	5.13E+02	2.97E+01	1564.586076	1.10E+01	5.55E+03	0.531932	40.222222	122
	flat100	38	5.91E+00	0.00E+00	12.377588	0.00E+00	3.74E+01	0.832755	37	207
	flat25	47	1.21E+02	1.67E+01	710.295968	4.37E+00	4.89E+03	0.748976	28.744681	51
	flat50	44	5.91E+00	3.31E+00	9.672536	0.00E+00	3.91E+01	0.907084	30.204545	82
	mixed period10	49	1.28E+04	4.96E+01	13359.38617	2.26E+01	3.14E+04	0.272553	40.55102	71
	mixed period100	46	7.04E+01	0.00E+00	438.62715	0.00E+00	2.98E+03	0.846571	33.282609	81
	mixed period25	41	1.51E+04	1.72E+04	6740.905032	2.55E+01	2.23E+04	0.651292	30.560976	72
	mixed period50	46	5.39E+03	5.54E+03	3793.113598	7.02E+00	1.16E+04	0.788439	29.434783	71
	overload140	44	5.56E+00	0.00E+00	13.100812	0.00E+00	4.27E+01	0.841441	31.136364	67

Table 3: Summary of rt-app Failures

Setup	Total Trials	Failed Trials	Failure Rate (%)
Baremetal + Mainline + CFS	500	0	0.0
Baremetal + Mainline + DL	500	0	0.0
Baremetal + PREEMPT_RT + DL	500	103	20.6
Docker + Mainline + DL	500	59	11.8
Docker + PREEMPT_RT + DL	500	50	10.0

Table 4: Summary of Worst Observed cyclicttest Latency (μ s)

Workload	Baremetal + Mainline + CFS	Baremetal + Mainline + DL	Baremetal + PREEMPT_RT +DL	Docker + Mainline + DL	Docker + PREEMPT_RT + DL
bursty	4057	5702	47	8594	254
flat10	5426	5633	124	5894	122
flat25	5662	5422	46	5632	51
flat50	5720	5047	46	5641	82
flat100	4920	5414	64	5207	207
mixed_period10	5759	4570	57	5232	71
mixed_period25	6557	5124	65	4674	72
mixed_period50	5971	4872	48	5087	71
mixed_period100	6063	5476	39	5384	81
overload140	6056	4943	67	5319	67

8.2 SCRIPTS & CODE

```
{
  "tasks": {
    "bursty_task": {
      "instance": 1,
      "loop": -1,
      "policy": "SCHED_DEADLINE",
      "dl-runtime": 30000,
      "dl-period": 50000,
      "dl-deadline": 50000,
      "phases": {
        "light_phase": {
          "loop": 10,
          "run": 5000,
          "timer": { "ref": "unique", "period": 50000 }
        },
        "heavy_phase": {
          "loop": 3,
          "run": 30000,
          "timer": { "ref": "unique", "period": 50000 }
        }
      }
    }
  },
  "global": {
    "duration": 30,
    "calibration": "CPU0",
    "default_policy": "SCHED_DEADLINE",
    "pi_enabled": false,
    "lock_pages": false,
    "logdir": "./",
    "log_basename": "rt-app2",
    "gnuplot": true
  }
}
```

Figure 7: *bursty.json*

```

# For flat25, flat50, and flat100 dl-runtime was adjusted to take the
load from 10% here
# to 25%, 50%, and 100% for each respectively.
{
  "tasks" : {
    "thread0" : {
      "instance" : 1,
      "loop" : -1,
      "policy": "SCHED_DEADLINE",
      "dl-runtime": 5000,
      "dl-period": 100000,
      "dl-deadline": 100000,
      "run" : 5000,
      "sleep" : 0,
      "timer" : { "ref" : "unique", "period" : 100000 }
    },
    "thread1" : {
      "instance" : 1,
      "loop" : -1,
      "policy": "SCHED_DEADLINE",
      "dl-runtime": 5000,
      "dl-period": 100000,
      "dl-deadline": 100000,
      "run" : 5000,
      "sleep" : 0,
      "timer" : { "ref" : "unique", "period" : 100000 }
    }
  },
  "global" : {
    "duration" : 30,
    "calibration" : "CPU0",
    "default_policy" : "SCHED_DEADLINE",
    "pi_enabled" : false,
    "lock_pages" : false,
    "logdir" : "./",
    "log_basename" : "rt-app2",
    "gnuplot" : true
  }
}

```

Figure 8: flat10.json

```

# For mixed_period25, mixed_period50, and mixed_period100 dl-runtime
was adjusted to take # the load from 10% here to 25%, 50%, and 100% for
each respectively.
{
  "tasks": {
    "fast_task": {
      "instance": 1,
      "loop": -1,

      "policy": "SCHED_DEADLINE",
      "dl-runtime": 1000,
      "dl-period": 25000,
      "dl-deadline": 25000,
      "run": 1000,
      "timer": { "ref": "unique", "period": 25000 }
    },
    "medium_task": {
      "instance": 1,
      "loop": -1,

      "policy": "SCHED_DEADLINE",
      "dl-runtime": 3000,
      "dl-period": 100000,
      "dl-deadline": 100000,
      "run": 3000,
      "timer": { "ref": "unique", "period": 100000 }
    },
    "slow_task": {
      "instance": 1,
      "loop": -1,

      "policy": "SCHED_DEADLINE",
      "dl-runtime": 6000,
      "dl-period": 200000,
      "dl-deadline": 200000,
      "run": 6000,
      "timer": { "ref": "unique", "period": 200000 }
    }
  },
  "global": {
    "duration": 30,
    "calibration": "CPU0",
    "default_policy": "SCHED_DEADLINE",
    "pi_enabled": false,
    "lock_pages": false,
    "logdir": "./",
    "log_basename": "rt-app2",
    "gnuplot": true
  }
}

```

Figure 9: mixed_period10.json

```
{
  "tasks": {
    "thread0": {
      "instance": 1,
      "loop": -1,

      "policy": "SCHED_DEADLINE",
      "dl-runtime": 70000,
      "dl-period": 100000,
      "dl-deadline": 100000,
      "run": 70000,
      "timer": { "ref": "unique", "period": 100000 }
    },
    "thread1": {
      "instance": 1,
      "loop": -1,

      "policy": "SCHED_DEADLINE",
      "dl-runtime": 70000,
      "dl-period": 100000,
      "dl-deadline": 100000,
      "run": 70000,
      "timer": { "ref": "unique", "period": 100000 }
    }
  },
  "global": {
    "duration": 30,
    "calibration": "CPU0",
    "default_policy": "SCHED_DEADLINE",
    "pi_enabled": false,
    "lock_pages": false,
    "logdir": "./",
    "log_basename": "rt-app2",
    "gnuplot": true
  }
}
```

Figure 10: overload140.json

```
#!/usr/bin/env bash
set -euo pipefail

sudo echo "Enter sudo pass to start"
OUTDIR="./results_docker-preempt-deadline"
WORKLOAD_DIR="./workloads"
TRIALS=50

mkdir -p "$OUTDIR"

CSV="$OUTDIR/results_docker-preempt-deadline.csv"

if [[ ! -f "$CSV" ]]; then
    echo
    "workload,trial,run_dir,rtapp_status,cyclictest_max_us,rtapp_miss_rate,rtapp_jitter_s
    tddev_us,rtapp_avg_wu_latency_us" > "$CSV"
fi

extract_cyclictest_max() {
    awk '
    {
        for (i = 1; i <= NF; i++) {
            if ($i == "Max:") {
                if ($(i+1) > max) max = $(i+1)
            }
        }
    }
    END {
        if (max == "") print "NA";
        else print max;
    }' "$1"
}

extract_rtapp_stats() {
    local logfile="$1"

    awk '
    /^#/ { next }
    NF < 11 { next }
    {
        total++

        slack = $8
        jitter = $11
        wu_lat = $11

        if (slack < 0) misses++

        n++
        delta = jitter - mean
        mean += delta / n
        m2 += delta * (jitter - mean)

        lat_total += wu_lat
    }
    END {
        if (total == 0 || n < 2) {
            print "NA,NA,NA"
        } else {
            miss_rate = misses / total
            stddev = sqrt(m2 / (n - 1))
            avg_latency = lat_total / total
            print miss_rate "," stddev "," avg_latency
        }
    }' "$logfile"
}
```

```

}

for WORKLOAD in "$WORKLOAD_DIR"/*.json; do
    WORKLOAD_NAME="$(basename "$WORKLOAD" .json)"
    WORK_DIR="$OUTDIR/${WORKLOAD_NAME}"

    for TRIAL in $(seq 1 "$TRIALS"); do
        RUN_ID="$(date +%Y-%m-%d %H:%M:%S)"
        RUNDIR="$WORK_DIR/${WORKLOAD_NAME}_trial${TRIAL}_${RUN_ID}"
        mkdir -p "$RUNDIR"

        echo "Now running $WORKLOAD_NAME trial $TRIAL..."

        cp "$WORKLOAD" "$RUNDIR/workload.json"

        sudo cyclicttest -q -t1 -p80 -i1000 -D10s > "$RUNDIR/cyclicttest.txt"
        CYCLIC_MAX="$(extract_cyclicttest_max "$RUNDIR/cyclicttest.txt")"

        pushd "$RUNDIR" > /dev/null

        if sudo rt-app workload.json > rtapp_stdout.txt 2> rtapp_stderr.txt; then
            RTAPP_STATUS="OK"
        else
            RTAPP_STATUS="FAILED"
        fi
        RTAPP_LOG="$(find . -maxdepth 1 -name 'rt-app2*.log' | head -n 1 || true)"

        popd > /dev/null

        if [[ -n "$RTAPP_LOG" ]]; then
            RTAPP_LOG="$RUNDIR/$(basename "$RTAPP_LOG")"
            RTAPP_STATS="$(extract_rtapp_stats "$RTAPP_LOG")"
        else
            RTAPP_STATS="NA,NA,NA"
        fi

        echo "$WORKLOAD_NAME,$TRIAL,$RUNDIR,$RTAPP_STATUS,$CYCLIC_MAX,$RTAPP_STATS" >>
"$CSV"

        echo "Finished $WORKLOAD_NAME trial $TRIAL"
        echo "-----"

    done
done

echo "All experiments complete."
echo "Results saved to $CSV"

```

Figure 11: experiment.sh

```
FROM debian:trixie

RUN apt-get update && apt-get install -y \
    rt-tests \
    rt-app \
    procps \
    util-linux \
    gawk \
    bash \
    coreutils \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /experiment
```

Figure 12: Dockerfile

```
sudo docker run --rm -it \
    --name rtos-exp \
    --privileged \
    --cap-add SYS_NICE \
    --security-opt seccomp=unconfined \
    --ulimit rtprio=99 \
    --ulimit memlock=-1 \
    --pid=host \
    --network=none \
    -v "$PWD:/experiment" \
    -v /dev:/dev \
    rtos-exp
```

Figure 13: docker_run.sh